

CI/CD & Git

From Zero to Viva-Ready

Continuous Integration & Delivery, pipelines, Git essentials, GitHub Actions, Jenkins, Docker basics + 25 viva questions

Quick-scan guide for SQA interviews and vivas

Companion to: [API Testing Guide](#) · [Selenium Guide](#) · [Playwright Guide](#) · [JMeter Guide](#) · [SQA A-Z Guide](#)

PART 1 — What problem does CI/CD solve?

Old way: developers work separately for weeks, merge everything at the end, and spend days fixing conflicts and broken builds ("integration hell"), then deploy manually and pray. CI/CD says: **integrate tiny changes constantly, test every single change automatically, and make releasing boring and push-button**. For QA, this is huge: your automated tests become the gate every change must pass.

Term	Meaning	One-liner
CI — Continuous Integration	Every code push is automatically built and TESTED	Merge small, test always
CD — Continuous Delivery	Every change that passes tests is automatically made READY to release; a human clicks 'deploy'	Always releasable, human decides
CD — Continuous Deployment	Passing changes go to production AUTOMATICALLY, no human click	Code to customer with zero clicks

Viva trap: Delivery vs Deployment — both are 'CD'. Delivery = ready to ship, human approves. Deployment = ships itself. Say this distinction unprompted and you score points.

What is a pipeline?

An automated assembly line for code. Typical stages: **Checkout code** → **Build** → **Unit tests** → **Integration/API tests** → **Deploy to staging** → **E2E/UI tests** → **Deploy to production**. If any stage fails, the line stops and the team is alerted — bad code never travels further. A **quality gate** is a rule that blocks progress (e.g. "all tests green", "coverage above 80%").

PART 2 — Git in 10 minutes (the foundation)

Git is a version control system — a save-game history for code. Every commit is a snapshot you can return to. GitHub is a website that hosts Git repositories and adds collaboration (pull requests, reviews, Actions).

Command / term	Meaning
git init / clone	Start a new repo / copy an existing one
git add → git commit	Stage changes → save a snapshot with a message
git push / pull	Send commits to GitHub / fetch others' commits
branch	A parallel line of work (feature-x) that doesn't disturb main
merge / pull request (PR)	Bring a branch into main; a PR is the review request before merging — where CI runs and teammates review
merge conflict	Two people changed the same lines — Git asks a human to choose; you edit, add, commit
git status / log / diff	What changed / history / exact line changes
.gitignore	Files Git must NOT track (reports, node_modules, secrets)

Your war story (use it in the viva!): "I once accidentally ran git add -A in my home folder and nearly staged my whole user profile — I caught it because I read git status before committing, reset, and re-initialized in the correct project folder. Lesson: always check git status before committing." Real mistakes told well = credibility.

PART 3 — GitHub Actions (what my project uses)

GitHub Actions is GitHub's built-in CI/CD. You describe the pipeline in a YAML file inside the repo at `.github/workflows/`, and GitHub runs it on its own servers. The vocabulary:

Concept	Meaning	In MY project
Workflow	One pipeline = one YAML file	api-tests.yml
Trigger (on:)	What starts it	push, pull_request, nightly cron (0 2 * * *), manual workflow_dispatch
Job	A group of steps on one machine	api-tests
Runner	The machine that executes the job	ubuntu-latest (GitHub-hosted)
Step	One command or reusable action	checkout → setup-node → install newman → run suite
Action	A reusable step from the marketplace	actions/checkout@v4, upload-artifact@v4
Artifact	A file saved from the run	Newman HTML report
Secrets	Encrypted variables (passwords, API keys) — never hard-code credentials in YAML	(demo creds are public, so none needed)

```
name: API Tests (Newman)
on: [push, pull_request]
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g newman
      - run: newman run postman/collection.json -e postman/env.json
```

Read it aloud: "On every push, on an Ubuntu machine: check out my code, install Newman, run my tests." That's a pipeline.

PART 4 — Jenkins & Docker (know the essentials)

Jenkins

The classic self-hosted CI server — you install and maintain it on your own machine/server. Pipelines are defined in a **Jenkinsfile** (Groovy) with stages; extended by thousands of plugins; jobs can be triggered by webhooks, schedules (cron), or manually. Difference from GitHub Actions in one line: **Actions is hosted-and-ready inside GitHub; Jenkins is self-hosted, infinitely customizable, and you maintain it yourself.** Many companies still run Jenkins — know the words: job, build, stage, agent/node, Jenkinsfile, plugin, webhook.

Docker (just enough for QA)

Docker packs an app plus everything it needs into a **container** — a lightweight box that runs identically on any machine. Kills "works on my machine" bugs. QA relevance: CI runners often execute tests inside containers; Selenium Grid and databases are commonly spun up as containers; an **image** is the recipe, a **container** is a running instance of it.

Where QA lives in the pipeline

- **Unit tests** (developers) run first — fastest feedback.
- **API tests** (my Newman suite) next — fast, stable, catch logic breaks.
- **UI/E2E tests** (Selenium/Playwright) after deploy to a test environment — slowest, run fewer.
- This ordering is the **test pyramid** applied to pipelines: cheap fast tests first, expensive slow tests last.
- **Shift-left** = this whole philosophy: test as early as possible, every single change.

PART 5 — 25 Viva Questions with Answers

Q: What is CI?

A: Continuous Integration — every push is automatically built and tested, so broken code is caught within minutes of being written.

Q: What is CD? Both meanings?

A: Continuous Delivery: every passing change is automatically made release-ready; a human clicks deploy. Continuous Deployment: it deploys to production automatically with no click.

Q: What is a pipeline?

A: An automated sequence of stages every change passes through — build, test layers, deploy — that stops the moment any stage fails.

Q: What is a quality gate?

A: A pass/fail rule that blocks the pipeline — e.g. all tests green, coverage threshold met, no critical static-analysis issues.

Q: Why does CI matter for QA specifically?

A: It turns my automated tests into an always-on regression safety net — every developer change must pass MY tests before it can merge. QA stops being a phase and becomes a gate.

Q: What is Git? GitHub?

A: Git = version control, a snapshot history of code. GitHub = a hosting service for Git repos plus collaboration: PRs, reviews, and Actions CI.

Q: What is a branch and why use one?

A: A parallel line of work. Features are built on branches so main always stays stable and releasable.

Q: What is a pull request?

A: A request to merge a branch into main — the place where teammates review the code and CI automatically runs the tests before merging is allowed.

Q: What is a merge conflict and how do you fix it?

A: Two changes touched the same lines; Git can't choose. You open the file, pick/combine the right version between the conflict markers, then add and commit.

Q: git fetch vs git pull?

A: fetch downloads remote changes without touching your work; pull = fetch + merge into your branch.

Q: What is .gitignore for?

A: Listing files Git must never track — generated reports, node_modules, environment files with secrets. My repo ignores Newman HTML reports because they're build artifacts, not source.

Q: What is GitHub Actions?

A: GitHub's built-in CI/CD — pipelines defined as YAML workflows in .github/workflows/, executed on GitHub-hosted runners.

Q: Explain workflow, job, step, runner.

A: Workflow = the whole pipeline file; job = a group of steps on one machine; step = one command or reusable action; runner = the machine executing it.

Q: What triggers can start a workflow?

A: push, pull_request, schedule (cron), workflow_dispatch (manual button), and more. My project uses all four.

Q: What is a cron schedule?

A: A time pattern for automatic runs. '0 2 * * *' = every day at 02:00 UTC — my nightly regression.

Q: What is an artifact?

A: A file the pipeline saves from a run — my workflow uploads the Newman HTML report so anyone can download the results.

Q: What are secrets in CI?

A: Encrypted variables (API keys, passwords) stored in repo settings and injected at runtime — credentials must never be hard-coded in YAML or committed to the repo.

Q: What happens when tests fail in the pipeline?

A: The job fails, the commit gets a red X, merging is blocked (with branch protection), and the team is notified. I open the logs/report, find the failing assertion, reproduce locally, and either file a bug or fix the test.

Q: What is Jenkins? How is it different from Actions?

A: The classic self-hosted CI server with pipelines defined in a Jenkinsfile and a huge plugin ecosystem. Actions is hosted inside GitHub with zero maintenance; Jenkins you host, customize, and maintain yourself.

Q: What is a Jenkinsfile?

A: A Groovy file in the repo defining the pipeline stages — same role as a GitHub Actions YAML.

Q: What is Docker and why does QA care?

A: Containers package an app with all dependencies so it runs identically everywhere. CI jobs and test infra (Selenium Grid, test DBs) commonly run in containers — bye-bye 'works on my machine'.

Q: Image vs container?

A: Image = the frozen recipe; container = a running instance of that image. One image, many containers.

Q: Where do different test types sit in a pipeline?

A: Test pyramid order: unit tests first (fastest), then API tests, then UI/E2E after deploying to a test environment (slowest, fewest). Fast feedback first.

Q: What is shift-left testing?

A: Moving testing earlier: test every change at commit time instead of a big phase at the end. CI is shift-left made real.

Q: Describe YOUR pipeline.

A: GitHub Actions workflow triggered by push, PR, nightly cron, and manually. It checks out the repo on ubuntu-latest, sets up Node 20, installs Newman + htmlextra, runs my 15-request/41-assertion Postman suite against Restful-Booker, publishes JUnit results as a check, and uploads the HTML report as an artifact.

Last-minute quick scan

If they say...	You say...
CI?	Every push auto-built and auto-tested.
Delivery vs Deployment?	Ready-to-ship with human click vs ships itself.
Pipeline?	Build → test layers → deploy; fails fast at any red stage.
PR?	Review + CI gate before merging to main.

If they say...	You say...
Workflow/job/step/runner?	Pipeline file / machine-group / command / machine.
Artifact / secret?	Saved output file / encrypted credential.
Jenkins vs Actions?	Self-hosted & plugin-rich vs GitHub-hosted & zero-maintenance.
Docker?	Same environment everywhere; image = recipe, container = running box.
Your pipeline?	Push/PR/nightly/manual → Newman suite → JUnit check + HTML artifact.

Strongest card you hold: you don't just know CI/CD theory — your repo HAS a working pipeline. Open the Actions tab in the interview and show a green run.